GUÍA DE ESTUDIO · PASO A PASO

Firebase explicado fácil

Cómo conectamos nuestra app de barbería a un proyecto nuevo de Firebase

> **IDEA**
> No necesitás saber programar para entender esta guía. **Solo imaginá que Firebase es una biblioteca mágica en internet** que guarda la información de tu app aunque apagues el teléfono.

Proyecto	BarberiaClaseA
Identificador	barberiaclasea-5f66b
Tipo de app	Web (la usa Expo con JavaScript)
Fecha	Octubre de 2026

Contenido

Cada número es una parada del recorrido. Podés leerlo en orden o buscar la parte que necesites.

1	La idea en 30 segundos	3
2	Diccionario de palabras raras	3
3	Qué hicimos en este proyecto	4
4	Crear el proyecto en Firebase	5
5	Conectar la app con el archivo .env	5
6	Las reglas: el guardia de seguridad	6
7	La colección productos	8
8	Probar que todo funciona	9
9	Si algo falla	10
10	Reglas de oro	11
11	Repaso: mini examen	11
	Apéndice: datos del proyecto	12

> **TRUCO**
> Si es la primera vez que ves Firebase, leé las secciones 1, 2 y 3. Ahí está el corazón de todo. Las demás son el “cómo se hace”.

1. La idea en 30 segundos

Imaginate que tu app es una **libreta** que vive dentro del teléfono. La libreta sirve para anotar usuarios y productos de una barbería.

El problema es que la libreta sola no alcanza: si se rompe el teléfono, se pierde todo. Por eso usamos **Firebase**, que es como una **biblioteca mágica en internet**.

Firebase guarda la información por nosotros. Aunque apagues la computadora, los datos siguen ahí. Y cualquier teléfono con internet puede pedirlos.



Ahora las tres piezas importantes, con nombres de cosas que ya conocés:

1. **Authentication** es la **puerta de entrada**. Ahí se registran y entran los usuarios con su correo y contraseña.
2. **Firestore** es el **armario de datos**. Tiene cajones llamados **colecciones** y, dentro, fichas llamadas **documentos**. Nuestro cajón principal se llama `productos`.
3. **Las reglas** son el **guardia de seguridad**. Deciden quién puede mirar o tocar el armario. Sin reglas, cualquiera podría entrar.

> **IDEA**
> Cuando te registrás, la app manda tus datos a Firebase. Cuando abrís la pantalla de Productos, la app le pregunta a Firestore: "¿qué productos hay?". Firestore responde y la app los muestra en una lista.

2. Diccionario de palabras raras

No hace falta memorizar todo. Volvé a esta tabla cuando veas una palabra que no entiendas.

Palabra	Qué es, en simple
Firebase	La biblioteca mágica en internet. Guarda datos, usuarios y más.
Proyecto	Tu cuaderno dentro de la biblioteca. Todo lo de la barbería vive ahí.
Credenciales	La dirección y la llave para encontrar tu cuaderno.
API Key	Un carnet que dice a qué proyecto pertenece la app. No es una contraseña secreta.
Authentication	La puerta donde los usuarios se registran y entran.

Palabra	Qué es, en simple
Firestore	El armario de datos. Guarda la información en cajones y fichas.
Colección	Un cajón con muchas fichas del mismo tipo. Ejemplo: productos.
Documento	Una ficha dentro del cajón. Ejemplo: un producto con su nombre y precio.
Reglas	El guardia que decide quién puede leer o escribir en el armario.
Archivo .env	Una mochila privada donde guardás las llaves, fuera del código.
Expo / Expo Go	La herramienta que muestra tu app en el teléfono mientras la probás.
SDK	El kit de herramientas que usa la app para hablar con Firebase.

3. Qué hicimos en este proyecto

Cambiamos la app de un Firebase viejo a uno nuevo. Es como **mudarse de casa**: la app es la misma, pero ahora sus datos viven en otra dirección.

Proyecto nuevo: `barberiaclasea-5f66b`

- ✔️ Creamos el proyecto en la consola de Firebase.
- ✔️ Registramos la app como Web, que es la que entiende Expo.
- ✔️ Activamos el acceso con correo y contraseña.
- ✔️ Creamos la base de datos Firestore.
- ✔️ Publicamos las reglas de seguridad.
- ✔️ Guardamos las seis credenciales en el archivo .env.
- ☐ Crear el primer producto desde la app (lo hacés vos en la sección 7).

> **LISTO**
> El proyecto viejo (`barberia-clase-a-4649d`) ya no se usa. La app ahora habla con el proyecto nuevo. Los usuarios y productos del proyecto viejo **no se mudan solos**: la casa nueva empieza vacía.

4. Crear el proyecto en Firebase

Esta parte se hace una sola vez, desde una computadora con internet. Necesitás una cuenta de Google.

1. Entrá a `console.firebase.google.com` e iniciá sesión con tu cuenta de Google.

2. Pulsá **Crear proyecto**. Escribí un nombre, por ejemplo `barberiaclasea`.
3. Google Analytics es opcional. Para este proyecto podés desactivarlo y pulsar **Crear proyecto**.
4. Cuando el proyecto esté listo, entrá a **Descripción general del proyecto**.
5. Pulsá el ícono `</>` (Web) para agregar una app.
6. Escribí un alias, por ejemplo `barberia-web`. No marques Firebase Hosting.
7. Pulsá **Registrar app**. Va a aparecer un cuadro con las credenciales. Copialo o dejá la página abierta.

> **OJO**
> Elegí siempre **Web (</>)**, no Android ni iOS. Nuestra app está hecha con Expo y usa el SDK de JavaScript, que se configura como una app web. Por eso no necesitamos el archivo `google-services.json`.

Los tres servicios que hay que activar

Servicio	Dónde está	Qué hay que hacer
Authentication	Menú izquierdo → Authentication	Comenzar → Método de acceso → Correo electrónico/contraseña → Habilitar → Guardar.
Firestore Database	Menú izquierdo → Firestore Database	Crear base de datos → Modo producción → Elegir ubicación → Crear.
Reglas	Firestore → pestaña Reglas	Pegar las reglas de la sección 6 → Publicar.

5. Conectar la app con el archivo .env

El archivo `.env` es como una **mochila privada**. Ahí guardamos las seis credenciales que la app necesita para encontrar el proyecto. La mochila queda en tu computadora y no se sube a GitHub.

Dentro del proyecto, abrí el archivo `.env` y completá una línea por dato:

```
EXPO_PUBLIC_FIREBASE_API_KEY=tu_api_key
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=tu_proyecto.firebaseapp.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=tu_proyecto
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=tu_proyecto.firebasestorage.app
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=123456789012
EXPO_PUBLIC_FIREBASE_APP_ID=1:123456789012:web:abc123
```

¿Qué guarda cada línea?

Dato del objeto firebaseConfig	Qué guarda	Ejemplo
`apiKey`	El carnet del proyecto.	AIzaSy...
`authDomain`	La dirección de la puerta de login.	miproyecto.firebaseapp.com
`projectId`	El nombre interno del proyecto.	miproyecto
`storageBucket`	El lugar para guardar archivos.	miproyecto.firebasestorage.app
`messagingSenderId`	Un número que identifica al proyecto.	123456789012
`appId`	El carnet de la app web.	1:123...:web:abc123

> **OJO**
> Copiá los valores **exactamente**. Sin comillas, sin comas y sin punto y coma al final. Cada dato va en su propia línea.

> **TRUCO**
> Si cambiás algo en `.env`, la app no lo ve hasta reiniciar Expo. Detené el servidor con `Ctrl+C` y arrancá de nuevo con `npx expo start -c`. La `-c` limpia la memoria vieja.

¿Perdiste las credenciales? No pasa nada. En la consola entrá a **Configuración del proyecto → Tus apps → Web → Configuración del SDK** y copiá el objeto `firebaseConfig` otra vez.

6. Las reglas: el guardia de seguridad

Firestore es un armario muy grande. Si no hubiera nadie cuidándolo, cualquier persona con internet podría leer o borrar los productos. Las **reglas** son el guardia que está en la puerta.

Nuestro guardia permite cuatro cosas, pero solo a usuarios que iniciaron sesión:

Acción	¿Qué significa?
read	Mirar la lista de productos.
create	Agregar un producto nuevo.
update	Cambiar el precio, el stock o el nombre.
delete	Borrar un producto.

La frase clave es `if request.auth != null`, que en simple significa: **"solo si el usuario está logueado"**.

Estas son las reglas del proyecto:

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // Solo usuarios autenticados pueden leer y escribir productos.
    match /productos/{productoId} {
      allow read: if request.auth != null;

      allow create: if request.auth != null
        && request.resource.data.nombre is string
        && request.resource.data.nombre.size() > 0
        && request.resource.data.nombre.size() <= 80
        && request.resource.data.categoria is string
        && request.resource.data.categoria.size() > 0
        && request.resource.data.precio is number
        && request.resource.data.precio >= 0
        && request.resource.data.stock is number
        && request.resource.data.stock >= 0
        && request.resource.data.activo is bool;

      allow update: if request.auth != null
        && request.resource.data.nombre is string
        && request.resource.data.nombre.size() > 0
        && request.resource.data.nombre.size() <= 80
        && request.resource.data.categoria is string
        && request.resource.data.categoria.size() > 0
        && request.resource.data.precio is number
        && request.resource.data.precio >= 0
        && request.resource.data.stock is number
        && request.resource.data.stock >= 0
        && request.resource.data.activo is bool;

      allow delete: if request.auth != null;
    }

    // Cualquier otra colección queda bloqueada.
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Cómo publicarlas:

1. En la consola, entrá a **Firestore Database** → pestaña **Reglas**.
2. Borrá lo que haya y pegá el texto de arriba.
3. Pulsá **Publicar**. Si el botón no aparece, es porque no hay cambios pendientes.

> **OJO**
> Si el guardia no está de turno, la app muestra el error `permission-denied`. No significa que Firebase esté roto: significa que las reglas no están publicadas o que no iniciaste sesión.

> **IDEA**
> La consola de Firebase es como el **dueño de la biblioteca**: puede crear o borrar datos sin pedirle permiso al guardia. Por eso podés cargar un producto desde la consola aunque las reglas sean estrictas.

7. La colección productos

En Firestore **no hay tablas**. Hay **cajones** (colecciones) con **fichas** (documentos). Es más flexible: dos fichas pueden tener campos distintos.

> **LISTO**
> No hace falta crear la colección a mano. **Se crea sola** la primera vez que guardás un producto desde la app. Es como estrenar un cajón: aparece cuando ponés la primera ficha.

Opción A: desde la app (recomendada)

1. Arrancá la app con `npx expo start -c` y abrila en tu teléfono.
2. Registrate o iniciá sesión con tu usuario.
3. Entrá a **Productos → Registrar producto**.
4. Completá nombre, categoría, precio y stock. Guardá.
5. En la consola, entrá a **Firestore → Datos**. Ya vas a ver la colección `productos`.

Opción B: desde la consola

Si querés verla antes de usar la app:

1. Entrá a **Firestore → Datos**.
2. Pulsá **Iniciar colección**.
3. ID de la colección: `productos` (todo minúscula, sin tilde).
4. ID del documento: dejá **ID automático**.
5. Agregá los campos de la tabla de abajo y pulsá **Guardar**.

Campo	Tipo	¿Obligatorio?	Ejemplo
`nombre`	Texto (string)	Sí	Corte clásico
`categoria`	Texto (string)	Sí	Cortes
`precio`	Número (number)	Sí	8000
`stock`	Número (number)	Sí	10
`activo`	Booleano (true/false)	Sí	true

Campo	Tipo	¿Obligatorio?	Ejemplo
`marca`	Texto (string)	No	Barber
`descripcion`	Texto (string)	No	Corte con tijera
`imagenUrl`	Texto (string)	No	(vacío)
`creadoEn`	Fecha y hora	La pone la app	—
`actualizadoEn`	Fecha y hora	La pone la app	—

TRUCO

El nombre del cajón tiene que ser exacto: `productos`. Minúscula, sin acento y sin espacios. Si escribís `Productos` o `producto`, la app no lo va a encontrar.

IDEA

La app completa los campos que faltan con valores vacíos o cero. Por eso un documento puede tener solo los cinco campos obligatorios y igual va a funcionar.

8. Probar que todo funciona

Llegó el momento de la verdad. Seguí esta lista y marcá cada casilla:

- ☐ Abrí la terminal en la carpeta del proyecto y ejecutá `npx expo start -c`.
- ☐ Escaneá el QR con Expo Go (el teléfono y la compu tienen que estar en la misma red Wi-Fi).
- ☐ Registrate con un correo y una contraseña que cumpla: 6 caracteres, una mayúscula, una minúscula y un número (por ejemplo, `Prueba123`).
- ☐ Comprobá que entraste a la pantalla de Inicio.
- ☐ Andá a Productos y cargá uno con nombre, categoría, precio y stock.
- ☐ En la consola, mirá Authentication → Usuarios y Firestore → Datos → productos.
- ☐ Cerrá sesión, volvé a entrar y verificá que el producto sigue ahí. También cerrá y abrí Expo Go: la sesión tiene que conservarse.

LISTO

Si ves tu producto en la consola, **¡felicitaciones!** La app, Firebase y Firestore se están hablando bien. Ya tenés la base de la barbería funcionando.

¿Y si quiero editar o borrar?

La pantalla de Productos ya permite editar, cambiar el stock, activar o desactivar y eliminar. Cada cambio se guarda en el mismo documento de Firestore. No hace falta tocar la consola.

9. Si algo falla

Los errores no son un castigo: son pistas. Buscá tu mensaje en esta tabla.

Lo que ves	Qué significa	Qué hacer
Falta configurar Firebase	El archivo .env no existe o tiene algún valor vacío.	Revisá que estén las seis líneas completas y reiniciá con npx expo start -c.
Clave API no válida	La API Key está mal copiada o es de otro proyecto.	Copiala de nuevo desde Configuración del proyecto → Tus apps → Web.
permission-denied	El guardia no te deja pasar: reglas sin publicar o sesión cerrada.	Publicá las reglas y asegurate de haber iniciado sesión.
failed-precondition	Firestore no está creado en ese proyecto.	Creá la base de datos en Firestore Database → Crear base de datos.
unavailable	Se cortó internet o la red no llega a Firebase.	Revisá tu conexión y volvé a intentar.
El producto ya no existe	Otro dispositivo lo borró mientras lo mirabas.	Volvé a la lista y actualizá.
El QR no conecta	El teléfono y la compu no se ven en la red local.	Ejecutá npm run start:tunnel y escaneá el QR nuevo.

> **TRUCO**
> Si el error no está en la tabla, copiá el texto completo y pegalo en el chat del proyecto. Casi siempre el mensaje dice exactamente qué archivo o qué permiso falta.

10. Reglas de oro

- ★ **La API Key no es una contraseña.** Sirve para identificar el proyecto. Lo que de verdad protege los datos son las reglas de Firestore.
- ★ **No compartas el archivo .env.** Tiene las llaves de tu proyecto y está excluido de Git a propósito.
- ★ **Proyecto nuevo = casa vacía.** Los usuarios y productos del proyecto anterior no se mudan solos. Hay que volver a crearlos.

- ★ **Si cambiás el .env, reiniciá con -c.** La app lee las credenciales al arrancar; si no reiniciás, sigue usando las viejas.
- ★ **Probá siempre con sesión iniciada.** El guardia deja pasar solo a usuarios logueados.
- ★ **La colección se llama productos.** Minúscula, sin tilde y sin espacios. Ese nombre está escrito en el código.

11. Repaso: mini examen

Respondé mentalmente. Después mirá las respuestas, que están al final y un poquito escondidas.

1. ¿Qué es Firestore y para qué lo usamos?
2. ¿Quién puede leer los productos de nuestra app?
3. ¿Dónde se guardan las seis credenciales?
4. ¿Qué pasa si cambio el .env y no reinicio Expo?
5. ¿La colección productos se crea sola o hay que crearla a mano?
6. ¿Cuál es la diferencia entre una colección y un documento?
7. ¿Por qué la app usa una app Web de Firebase y no una de Android?

> **IDEA**
>
> **Respuestas:** 1) Es el armario de datos; guarda los productos. 2) Solo los usuarios con sesión iniciada. 3) En el archivo .env. 4) La app sigue usando las credenciales viejas. 5) Se crea sola al guardar el primer producto. 6) La colección es el cajón; el documento es una ficha dentro. 7) Porque Expo usa el SDK de JavaScript, que se configura como una app Web.

Apéndice: datos del proyecto

Para tener a mano. Las llaves completas están en el archivo `.env` y se pueden volver a copiar desde la consola cuando quieras.

Dato	Valor
Nombre visible	BarberiaClaseA
projectId	barberiaclasea-5f66b
Tipo de app registrada	Web (</>)
Colección principal	`productos`
Consola	console.firebase.google.com
Archivo de llaves	`.env`
Archivo de reglas	`docs/firestore.rules`
Documentación extra	`docs/productos-firestore.md`

Fin de la guía

Ahora ya sabés qué hace cada pieza. El resto es práctica.